

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN KHOA HỌC MÁY TÍNH

BÁO CÁO MÔN HỌC
BIỂU DIỄN TRI THỨC

**Đề tài: HLG LLM UTILS – AN ADVANCED GRAPH RAG
FRAMEWORK WITH MULTI-AGENT ORCHESTRATION,
HIERARCHICAL LEIDEN COMMUNITY DETECTION, AND
CYPHER QUERY SELF-CORRECTION**

Giảng viên hướng dẫn: PGS. TS. Đỗ Văn Nhơn

Học viên thực hiện: 1. Nguyễn Hoài Nam – MSHV: 25C01014
2. Phạm Cung Lê Nhân Vũ – MSHV: 25C01049

Chuyên ngành: Khoa học Dữ liệu

Khóa: K35

Tóm Tắt Báo Cáo

Các hệ thống **Naive RAG** (Retrieval-Augmented Generation) dựa trên phân đoạn văn bản phẳng (flat chunking) và tìm kiếm độ tương đồng vector nhúng (**Cosine Similarity**) gặp phải hai hạn chế cốt lõi trong lý thuyết và thực tiễn Biểu diễn Tri thức: (1) Thất bại trong các truy vấn mang tính vĩ mô, tổng hợp trên toàn thể tài liệu (**Global Query**) do đứt gãy cấu trúc ngữ cảnh; và (2) Bất lực trong suy luận đa bước (**Multi-Hop Reasoning**) qua các chuỗi quan hệ thực thể phức tạp.

Báo cáo này trình bày toàn diện công trình nghiên cứu, thiết kế kiến trúc và cài đặt hệ thống **HLG LLM Utils**, một khung làm việc **Graph RAG** nâng cao kết hợp chặt chẽ giữa Đồ thị Tri thức Thuộc tính (**Property Knowledge Graph**) trên cơ sở dữ liệu Neo4j và Mô hình Ngôn ngữ Lớn (**LLM**). Hệ thống tập trung triển khai và làm chủ bốn động cơ truy xuất – suy luận cốt lõi:

1. **Mô hình hóa Đồ thị Tri thức Thuộc tính (Property Knowledge Graph Formalization):** Chuẩn hóa toán học và tự động trích xuất các bộ ba thực thể, quan hệ ngữ nghĩa, thuộc tính và văn bản nguồn từ tài liệu phi cấu trúc vào Neo4j với cơ chế nạp dữ liệu Idempotent MERGE.
2. **Phân hoạch Cộng đồng Phân cấp Leiden (Hierarchical Leiden Community Detection):** Áp dụng thuật toán tối ưu hóa độ modun đa cấp độ trên thư viện `igraph` và `leidenalg`, tự động tạo lập cây phân cấp cộng đồng và sinh bản tóm tắt ngữ nghĩa song song bằng LLM để phục vụ thuật toán tìm kiếm toàn cục theo mô hình **Map-Reduce**.
3. **Máy trạng thái LangGraph Tự Sửa Lỗi Truy vấn Cypher (Self-Correcting Cypher State Machine):** Xây dựng đồ thị trạng thái có khả năng chuyển ngữ tự nhiên sang Cypher chuẩn xác, kết hợp vòng lặp phản hồi 3 bước dựa trên lược đồ đồ thị thực và log lỗi từ Neo4j.
4. **Khám phá Đồ thị Đa tác thể ARK (Adaptive Retriever of Knowledge):** Triển khai thuật toán khám phá quỹ đạo đa tác thể theo nghiên cứu ACL 2026, sử dụng dung hợp thứ hạng bỏ phiếu (**Voting Rank Fusion**) để thích ứng linh hoạt giữa độ rộng và độ sâu khi duyệt đồ thị.

Báo cáo trình bày chi tiết nền tảng lý thuyết, công thức toán học, thiết kế lược đồ cơ sở dữ liệu, kiến trúc phần mềm Monorepo (`uv workspaces`), và các thuật toán đã được cài đặt thực tế trong mã nguồn dự án.

Từ khóa: Graph RAG, Knowledge Representation, Property Knowledge Graph, Hierarchical Leiden Algorithm, Cypher Self-Correction, Multi-Agent Systems, LangGraph, Adaptive Retriever of Knowledge, Monorepo uv.

Mục lục

Tóm Tắt Báo Cáo	i
1 Giới Thiệu và Đặt Vấn Đề	1
1.1 Bối Cảnh Nghiên Cứu và Động Cơ	1
1.2 Các Điểm Nghẽn Cốt Lõi của Naive RAG	1
1.3 Mục Tiêu và Đóng Góp của Đề Tài	2
2 Cơ Sở Lý Thuyết và Các Khái Niệm Nền Tảng	3
2.1 Từ Mạng Ngữ Nghĩa đến Property Knowledge Graph	3
2.1.1 Mạng Ngữ Nghĩa (Semantic Networks) và RDF Triples	3
2.1.2 Đồ thị Tri thức Thuộc tính (Property Knowledge Graph)	3
2.2 Không Gian Vector và Truy Xuất Lai (Hybrid Retrieval)	3
2.3 Lý Thuyết Phân Hoạch Đồ Thị và Thuật Toán Leiden	4
2.3.1 Hàm Tối Ưu Độ Modun (Modularity)	4
2.3.2 Cơ Chế Khắc Phục của Thuật Toán Leiden So với Louvain	4
2.4 Khám Phá Đồ Thị Đa Tác Thể Thích Ứng (ARK)	4
3 Hình Thức Hóa Biểu Diễn Tri Thức và Thiết Kế Lược Đồ	5
3.1 Hình Thức Hóa Toán Học Property Knowledge Graph	5
3.2 Thiết Kế Lược Đồ Cơ Sở Dữ Liệu Neo4j	5
3.2.1 Cấu Trúc Thuộc Tính của Các Nút	5
3.3 Hệ Thống Chỉ Mục Chuyên Biệt trên Neo4j	6
4 Quy Trình Đánh Chỉ Mục và Trích Xuất Tri Thức	7
4.1 Quy Trình Tiền Xử Lý và Phân Đoạn Văn Bản	7
4.2 Trích Xuất Thực Thể và Quan Hệ bằng Structured LLM	7
4.3 Kỹ Thuật Nạp Dữ Liệu Cypher Idempotent MERGE	7
5 Phân Hoạch Cộng Đồng Phân Cấp và Tóm Tắt Ngữ Nghĩa	9
5.1 Thuật Toán Phân Hoạch Phân Cấp Đa Cấp Độ	9
5.2 Cài Đặt với Fallback giữa Neo4j GDS và Python igraph	9
5.3 Quy Trình Tóm Tắt Ngữ Nghĩa Song Song bằng LLM	10
6 Các Động Cơ Truy Vấn và Suy Luận Đồ Thị Nâng Cao	11
6.1 Tổng Quan Các Chế Độ Truy Vấn	11
6.2 Động Cơ Truy Vấn Cục Bộ (Local Search)	11
6.3 Động Cơ Truy Vấn Toàn Cục (Global Search - Map-Reduce)	11
6.4 Máy Trạng Thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher	12
6.5 Động Cơ Khám Phá Thích Ứng Đa Tác Thể ARK	12
6.5.1 Tập Công Cụ của Tác Thể ARK	13
6.5.2 Dung Hợp Thứ Hạng Bỏ Phiếu (Voting Rank Fusion)	13

7	Kiến Trúc Hệ Thống và Thiết Kế Monorepo Modular	14
7.1	Kiến Trúc Tổng Thể Monorepo UV Workspace	14
7.2	Phân Tích Chi Tiết Các Gói Thành Phần	14
7.3	Tích Hợp Backend FastAPI và Service Registry Lazy Singleton	15
8	Thảo Luận, Thách Thức và Hướng Phát Triển	16
8.1	Thảo Luận Kiến Trúc và Ưu Thế Thực Tiễn	16
8.2	Những Thách Thức Kỹ Thuật	16
8.3	Hướng Phát Triển Trong Tương Lai	16
9	Kết Luận và Phân Công Nhiệm Vụ	17
9.1	Kết Luận	17
9.2	Bảng Phân Công Nhiệm Vụ Nhóm	17
	Tài Liệu Tham Khảo	18

Danh sách hình vẽ

3.1	Lược đồ Cấu trúc Đồ thị Tri thức Neo4j trong HLG LLM Utils	6
5.1	Cấu trúc Cây Phân cấp Cộng đồng Sinh bởi Thuật toán Leiden	9
6.1	Đồ thị Trạng thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher	12
7.1	Sơ đồ Phụ thuộc Mô-đun trong uv Workspace Monorepo	14

Danh sách bảng

9.1	Bảng Phân Công Nhiệm Vụ Chi Tiết Giữa Các Thành Viên	17
-----	--	----

Chương 1

Giới Thiệu và Đặt Vấn Đề

1.1 Bối Cảnh Nghiên Cứu và Động Cơ

Trong lĩnh vực Trí tuệ Nhân tạo hiện đại, sự bùng nổ của các Mô hình Ngôn ngữ Lớn (**Large Language Models – LLMs**) đã mở ra khả năng hiểu và tạo sinh ngôn ngữ tự nhiên chưa từng có. Tuy nhiên, các mô hình LLM thuần túy luôn tiềm ẩn hai điểm yếu mang tính bản chất: hiện tượng tạo sinh thông tin sai lệch (**Hallucination**) và sự tĩnh tại của tri thức (tri thức đóng gói trong trọng số không được cập nhật theo thời gian thực). Để khắc phục các hạn chế này, kỹ thuật **Retrieval-Augmented Generation (RAG)** đã trở thành mô hình tiêu chuẩn nhằm tiếp cận nguồn tri thức bên ngoài [4].

Mặc dù vậy, thể hệ RAG truyền thống (**Naive RAG**) – vốn dựa trên việc chia nhỏ văn bản thành các đoạn phẳng (**Chunks**) có kích thước cố định và tìm kiếm qua độ tương đồng vector (**Vector Cosine Similarity**) – bộc lộ nhiều điểm nghẽn nghiêm trọng khi áp dụng vào các bài toán Biểu diễn Tri thức phức tạp.

1.2 Các Điểm Nghẽn Cốt Lõi của Naive RAG

Qua phân tích sâu về lý thuyết biểu diễn tri thức và không gian vector, chúng tôi xác định ba rào cản mang tính cốt tử của mô hình Naive RAG phẳng:

- Đứt gãy Ngữ cảnh Cấu trúc (Structural Context Fragmentation):** Việc cắt nhỏ tài liệu thành các khối văn bản $c_i \in \mathcal{C}$ theo kích thước cố định (ví dụ: 500 hoặc 1000 tokens) vô tình phá hủy các mối liên kết ngữ nghĩa nằm xuyên suốt tài liệu. Khi thực thể E_1 được định nghĩa ở đoạn c_1 và có quan hệ nhân quả với thực thể E_2 ở đoạn c_k , mối quan hệ này bị xóa bỏ trong không gian vector phẳng.
- Bất lực trong Suy luận Đa bước (Multi-Hop Reasoning Failures):** Xét bài toán truy vấn đòi hỏi suy luận bắc cầu:

$$E_{\text{start}} \xrightarrow{R_1} E_{\text{mid}} \xrightarrow{R_2} E_{\text{target}} \quad (1.1)$$

Khi người dùng đặt câu hỏi liên quan đến E_{start} và E_{target} , câu truy vấn q chỉ có độ tương đồng vector cao với các đoạn chứa E_{start} hoặc E_{target} , trong khi thực thể trung gian E_{mid} bị bỏ qua hoàn toàn. Kết quả là ngữ cảnh truy xuất không đủ điều kiện để LLM thực hiện suy luận logic.

- Điểm mù Truy vấn Toàn cục (Global Query Blindspots):** Đối với các câu hỏi mang tính tổng hợp vĩ mô (Macro-summarization), ví dụ: “*Các nhóm chủ đề chính được thảo luận trong tập văn bản này là gì?*” hoặc “*Tóm tắt cấu trúc hợp tác giữa các tổ chức*”, vector nhúng $\mathbf{q} \in \mathbb{R}^d$ của câu hỏi không có độ tương đồng cao với bất kỳ đoạn văn bản đơn lẻ nào. Naive RAG hoàn toàn thất bại trước các câu hỏi dạng này [2].

1.3 Mục Tiêu và Đóng Góp của Đề Tài

Để giải quyết triệt để các hạn chế trên, đề tài tập trung nghiên cứu, thiết kế kiến trúc và cài đặt thực tế hệ thống **HLG LLM Utils** – một khung làm việc **Graph RAG** toàn diện kết hợp Đồ thị Tri thức Thuộc tính Neo4j với các thuật toán suy luận tiên tiến.

Các đóng góp cụ thể của đề tài bao gồm:

- **Hình thức hóa Toán học và Lược đồ Đồ thị Tri thức:** Định nghĩa cấu trúc đồ thị thuộc tính $G = (\mathcal{V}, \mathcal{E}, \mathcal{P}, \mathcal{W})$, thiết kế lược đồ lưu trữ cho các nút `:Entity`, `:Chunk`, `:Document`, `:Community` và các quan hệ `:RELATED_TO`, `:BELONGS_TO`, `:CONTAINS_CHUNK`, `:MENTIONS`.
- **Quy trình Trích xuất Tri thức Idempotent:** Triển khai quy trình trích xuất thực thể/quan hệ có cấu trúc bằng mô hình Pydantic và LLM, lưu trữ bất đồng bộ vào Neo4j với cơ chế MERGE chống trùng lặp và theo dõi tài liệu nguồn.
- **Thuật toán Phân hoạch Cộng đồng Phân cấp Leiden:** Cài đặt thuật toán phân cụm Leiden tối ưu hóa độ modun trên `igraph` và `leidenalg`, tự động sinh tóm tắt cộng đồng đa cấp độ phục vụ tìm kiếm toàn cục theo mô hình **Map-Reduce**.
- **Máy trạng thái Tự Sửa Lỗi Cypher (LangGraph):** Xây dựng máy trạng thái tự động chuyển đổi ngôn ngữ tự nhiên sang truy vấn Cypher, có khả năng tự sửa lỗi cú pháp và lược đồ qua 3 vòng lặp phản hồi trực tiếp từ Neo4j log.
- **Khám phá Đồ thị Đa tác thể ARK:** Triển khai thuật toán khám phá thích ứng ARK với cơ chế dung hợp thứ hạng bỏ phiếu (**Voting Rank Fusion**) giúp cân bằng tối ưu giữa độ rộng và độ sâu khi duyệt đồ thị.
- **Kiến trúc Phần mềm Chuẩn Công Nghiệp:** Đóng gói toàn bộ hệ thống dưới dạng Monorepo đa gói (`uv workspaces`) phân tách module hoàn chỉnh, tích hợp Backend FastAPI và dịch vụ FastMCP.

Chương 2

Cơ Sở Lý Thuyết và Các Khái Niệm Nền Tảng

2.1 Từ Mạng Ngữ Nghĩa đến Property Knowledge Graph

Biểu diễn Tri thức (**Knowledge Representation – KR**) là một trụ cột nền tảng của Trí tuệ Nhân tạo, nghiên cứu phương thức hình thức hóa thế giới thực để máy tính có thể suy diễn và giải quyết vấn đề [3].

2.1.1 Mạng Ngữ Nghĩa (Semantic Networks) và RDF Triples

Mạng ngữ nghĩa truyền thống biểu diễn tri thức dưới dạng các nút (khái niệm) và cạnh có hướng (quan hệ), ví dụ: IS-A hoặc PART-OF. Trong khung W3C Semantic Web, tri thức được chuẩn hóa thành các bộ ba RDF (*Subject, Predicate, Object*):

$$\tau = (s, p, o) \in \mathcal{E} \times \mathcal{R} \times (\mathcal{E} \cup \mathcal{L}) \quad (2.1)$$

trong đó $\mathcal{E}$ là tập thực thể, $\mathcal{R}$ là tập quan hệ, và $\mathcal{L}$ là tập giá trị văn bản (literals). Mặc dù có tính chuẩn hóa cao, RDF gặp khó khăn khi cần gắn kèm các thuộc tính phụ trợ (metadata, trọng số, thời gian, nguồn trích xuất) lên bản thân cạnh quan hệ.

2.1.2 Đồ thị Tri thức Thuộc tính (Property Knowledge Graph)

Mô hình **Property Knowledge Graph (LPG)** khắc phục triệt để nhược điểm của RDF bằng cách cho phép cả đỉnh (nodes) và cạnh (relationships) đều có thể chứa nhãn định danh (*Labels*) và một tập hợp các thuộc tính dạng cặp khóa-giá trị (*Key-Value Properties*) [7]:

$$G = (\mathcal{V}, \mathcal{E}, \lambda_V, \lambda_E, \rho_V, \rho_E) \quad (2.2)$$

Mô hình LPG cho phép biểu diễn trực tiếp các thuộc tính phong phú như mô tả ngữ nghĩa (*description*), nhãn phân loại (*type*), độ tin cậy, vector nhúng (*embedding*), và danh sách các tài liệu nguồn chứa thực thể (*source_documents*).

2.2 Không Gian Vector và Truy Xuất Lai (Hybrid Retrieval)

Trong không gian biểu diễn liên tục, mỗi đoạn văn bản hoặc thực thể x được ánh xạ thành một vector đặc trưng nhiều chiều $\mathbf{e}_x \in \mathbb{R}^d$ thông qua mô hình Transformer mã hóa [6]:

$$\mathbf{e}_x = \text{Embed}(x) \in \mathbb{R}^d, \quad \|\mathbf{e}_x\|_2 = 1 \quad (2.3)$$

Độ tương đồng giữa câu truy vấn q và thực thể/đoạn văn bản x được tính theo hàm Cosine Similarity:

$$\text{Sim}(q, x) = \langle \mathbf{e}_q, \mathbf{e}_x \rangle = \sum_{i=1}^d e_{q,i} \cdot e_{x,i} \quad (2.4)$$

Hệ thống kết hợp phương pháp tìm kiếm từ khóa (**BM25 / Fulltext Index**) và tìm kiếm ngữ nghĩa vector (**Dense Vector Index**) theo chiến lược **Hybrid Search**, áp dụng công thức dung hợp thứ hạng tương hỗ (**Reciprocal Rank Fusion – RRF**) [1]:

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{Vector}, \text{Fulltext}\}} \frac{1}{k_{\text{rrf}} + \text{rank}_m(d)} \quad (2.5)$$

với hằng số làm mịn $k_{\text{rrf}} = 60$.

2.3 Lý Thuyết Phân Hoạch Đồ Thị và Thuật Toán Leiden

Phân hoạch đồ thị là bài toán phân chia tập đỉnh $\mathcal{V}$ thành các tập con không giao nhau $\mathcal{P} = \{C_1, C_2, \dots, C_k\}$ sao cho mật độ cạnh nội bộ bên trong mỗi cụm là cực đại và mật độ cạnh liên cụm là cực tiểu [8].

2.3.1 Hàm Tối Ưu Độ Modun (Modularity)

Độ modun Q của một phân hoạch đồ thị trên đồ thị vô hướng $G = (\mathcal{V}, \mathcal{E})$ có tổng trọng số $m = \frac{1}{2} \sum_{ij} A_{ij}$ được định nghĩa bởi Newman-Girvan:

$$Q = \frac{1}{2m} \sum_{i,j \in \mathcal{V}} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(C(i), C(j)) \quad (2.6)$$

trong đó A_{ij} là ma trận kề, k_i là bậc của đỉnh i , γ là tham số độ phân giải (*resolution parameter*), $C(i)$ là cộng đồng chứa đỉnh i , và $\delta(u, v)$ là hàm Kronecker ($\delta = 1$ nếu $u = v$, ngược lại $\delta = 0$).

2.3.2 Cơ Chế Khắc Phục của Thuật Toán Leiden So với Louvain

Thuật toán Louvain truyền thống có một điểm yếu nghiêm trọng là dễ tạo ra các cộng đồng bị ngắt kết nối nội bộ (*disconnected communities*) hoặc kết nối yếu. Thuật toán **Leiden** [8] giải quyết triệt để vấn đề này bằng quy trình 3 giai đoạn:

1. **Local Movement of Nodes:** Di chuyển cục bộ từng đỉnh đến cộng đồng lân cận nhằm tối đa hóa hàm chất lượng.
2. **Refinement of Partition:** Tinh chỉnh phân hoạch, trong đó các đỉnh có thể được nhóm lại thành các phân cụm con nhằm đảm bảo tính liên thông nội tại của từng cộng đồng.
3. **Aggregation of Graph:** Thu gọn đồ thị dựa trên các phân cụm đã tinh chỉnh và lặp lại quá trình cho đến khi đạt trạng thái tối ưu.

2.4 Khám Phá Đồ Thị Đa Tác Thể Thích Ứng (ARK)

Khung làm việc **ARK (Adaptive Retriever of Knowledge)** [5] là một mô hình tiên tiến được công bố tại ACL 2026, giải quyết bài toán duyệt đồ thị tri thức bằng cách điều phối song song nhiều tác thể (*Agents*) độc lập. Thay vì thực hiện tìm kiếm cố định theo chiều sâu (DFS) hay chiều rộng (BFS), ARK cho phép các tác thể tự động quyết định hành động tại mỗi bước: tìm kiếm toàn cục, duyệt lân cận, hoặc chọn lọc nút vào tập ứng viên câu trả lời.

Chương 3

Hình Thức Hóa Biểu Diễn Tri Thức và Thiết Kế Lược Đồ

3.1 Hình Thức Hóa Toán Học Property Knowledge Graph

Đồ thị tri thức trong hệ thống HLG LLM Utils được mô hình hóa toán học thành cấu trúc:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{P}_V, \mathcal{P}_E, \ell_V, \ell_E) \quad (3.1)$$

trong đó:

- $\mathcal{V} = \mathcal{V}_{\text{Entity}} \cup \mathcal{V}_{\text{Chunk}} \cup \mathcal{V}_{\text{Doc}} \cup \mathcal{V}_{\text{Comm}}$ là tập hợp các đỉnh phân hoạch theo các kiểu thực thể, phân đoạn văn bản, tài liệu gốc và cụm cộng đồng.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ là tập các cạnh có hướng biểu diễn quan hệ.
- $\ell_V : \mathcal{V} \rightarrow \{\text{Entity, Chunk, Document, Community}\}$ là hàm gán nhãn đỉnh.
- $\ell_E : \mathcal{E} \rightarrow \{\text{RELATED_TO, BELONGS_TO, CONTAINS_CHUNK, MENTIONS, PARENT_COMMUNITY}\}$ là hàm gán nhãn quan hệ.
- $\mathcal{P}_V(v)$ và $\mathcal{P}_E(e)$ là hàm ánh xạ gán thuộc tính cho đỉnh và cạnh.

3.2 Thiết Kế Lược Đồ Cơ Sở Dữ Liệu Neo4j

Lược đồ đồ thị trên Neo4j được thiết kế tối ưu cho cả tác vụ duyệt quan hệ và tìm kiếm ngữ nghĩa vector:

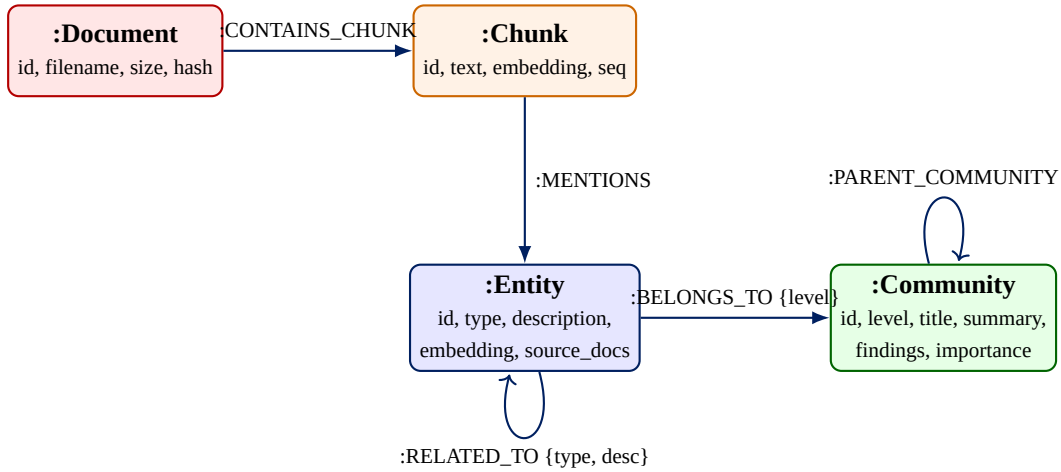
3.2.1 Cấu Trúc Thuộc Tính của Các Nút

1. Nút Thực Thể (:Entity):

- **id**: Khóa chính định danh duy nhất (chuỗi in hoa đã chuẩn hóa, ví dụ: "GRAPH_RAG").
- **type**: Phân loại thực thể (ví dụ: "CONCEPT", "ALGORITHM", "FRAMEWORK", "ORGANIZATION").
- **description**: Đoạn văn mô tả ngữ nghĩa chi tiết của thực thể.
- **embedding**: Vector đặc trưng ngữ nghĩa $\mathbf{e} \in \mathbb{R}^d$ ($d = 1536$ với OpenAI hoặc $d = 768$ với nomic-embed).
- **source_documents**: Mảng danh sách các tên tệp tài liệu nguồn chứa thực thể này.

2. Nút Cộng Đồng (:Community):

- **id**: Định danh cộng đồng theo cấp độ (ví dụ: "community_lvl0_14").
- **level**: Cấp độ trong cây phân cấp ($l \in \{0, 1, \dots, L_{\max} - 1\}$).



Hình 3.1: Lược đồ Cấu trúc Đồ thị Tri thức Neo4j trong HLG LLM Utils

- **title:** Tiêu đề tóm tắt ngắn gọn của cộng đồng (dưới 10 từ).
- **summary:** Đoạn văn tổng hợp các chủ đề và quan hệ chính trong cộng đồng do LLM sinh ra.
- **findings:** Mảng các phát hiện, sự kiện hoặc luận điểm quan trọng.
- **importance_score:** Điểm số đánh giá mức độ trọng yếu ($s \in [0.0, 1.0]$).
- **entity_count:** Số lượng thực thể thành viên thuộc cộng đồng.

3.3 Hệ Thống Chỉ Mục Chuyên Biệt trên Neo4j

Nhằm tối ưu hóa tốc độ truy vấn đa phương thức, hệ thống tự động khởi tạo các cấu trúc chỉ mục:

- **Vector Index trên Entity (entity_embeddings):**

```

1 CREATE VECTOR INDEX `entity_embeddings` IF NOT EXISTS
2 FOR (n:Entity) ON (n.embedding)
3 OPTIONS {indexConfig: {
4   `vector.dimensions`: $dimension,
5   `vector.similarity_function`: 'cosine'
6 }}
7

```

- **Vector Index trên Community (community_embeddings):**

```

1 CREATE VECTOR INDEX `community_embeddings` IF NOT EXISTS
2 FOR (c:Community) ON (c.embedding)
3 OPTIONS {indexConfig: {
4   `vector.dimensions`: $dimension,
5   `vector.similarity_function`: 'cosine'
6 }}
7

```

- **Fulltext Index trên Entity (entity_fulltext):**

```

1 CREATE FULLTEXT INDEX entity_fulltext IF NOT EXISTS
2 FOR (n:Entity) ON EACH [n.id, n.description, n.type]
3

```

Chương 4

Quy Trình Đánh Chỉ Mục và Trích Xuất Tri Thức

4.1 Quy Trình Tiền Xử Lý và Phân Đoạn Văn Bản

Quy trình nạp tài liệu (**Document Ingestion Pipeline**) được điều phối bởi `DocumentService` với các bước chặt chẽ:

1. Tiếp nhận tệp gốc từ MinIO S3 Storage và trích xuất văn bản thuần qua `llm-utils-parser` (hỗ trợ PDF qua PyMuPDF, Office, Code, và OCR Engine).
2. Thực hiện phân đoạn văn bản theo Token Window ($W = 1000$ tokens, độ gối đầu overlap $O = 200$ tokens).
3. Ghi nhận văn bản đoạn vào PostgreSQL và đồng thời tạo vector nhúng đoạn văn bản vào Qdrant / PGVector.

4.2 Trích Xuất Thực Thể và Quan Hệ bằng Structured LLM

Hệ thống sử dụng cơ chế ép kiểu cấu trúc Pydantic (`with_structured_output`) thông qua lớp `ExtractionResult`:

```
1 class EntityModel(BaseModel):
2     id: str
3     type: str = "CONCEPT"
4     description: str = ""
5
6 class RelationshipModel(BaseModel):
7     source: str
8     target: str
9     type: str = "RELATED_TO"
10    description: str = ""
11
12 class ExtractionResult(BaseModel):
13     entities: List[EntityModel] = Field(default_factory=list)
14     relationships: List[RelationshipModel] = Field(default_factory=list)
```

Prompt trích xuất được thiết kế đặc thù nhằm ép LLM trích xuất tri thức với định dạng chuẩn xác, tránh tạo sinh các thực thể chung chung vô nghĩa và đảm bảo mọi quan hệ đều có hướng rõ ràng.

4.3 Kỹ Thuật Nạp Dữ Liệu Cypher Idempotent MERGE

Để đảm bảo tính nhất quán dữ liệu khi chạy lại nhiều lần hoặc nạp tài liệu tăng cường, mã nguồn `indexing.py` cài đặt kỹ thuật MERGE bất biến:

```

1 MERGE (e:Entity {id: $id})
2 ON CREATE SET e.type = $type,
3               e.description = $description,
4               e.embedding = $embedding,
5               e.source_documents = [$source_doc]
6 ON MATCH SET e.description = COALESCE(e.description, $description),
7               e.embedding = $embedding,
8               e.source_documents = CASE
9                   WHEN e.source_documents IS NULL THEN [$source_doc]
10                  WHEN $source_doc IN e.source_documents THEN e.source_documents
11                  ELSE e.source_documents + [$source_doc]
12            END

```

Đối với quan hệ giữa các thực thể, câu truy vấn đảm bảo tính toàn vẹn tham chiếu và làm sạch tên quan hệ theo định dạng biểu thức chính quy [A-Z0-9_]:

```

1 MATCH (source:Entity {id: $source})
2 MATCH (target:Entity {id: $target})
3 MERGE (source)-[r:RELATED_TO]->(target)
4 ON CREATE SET r.description = $description, r.source_documents = [$source_doc]
5 ON MATCH SET r.description = COALESCE(r.description, $description),
6               r.source_documents = CASE
7                   WHEN r.source_documents IS NULL THEN [$source_doc]
8                   WHEN $source_doc IN r.source_documents THEN r.source_documents
9                   ELSE r.source_documents + [$source_doc]
10            END

```

Chương 5

Phân Hoạch Cộng Đồng Phân Cấp và Tóm Tắt Ngữ Nghĩa

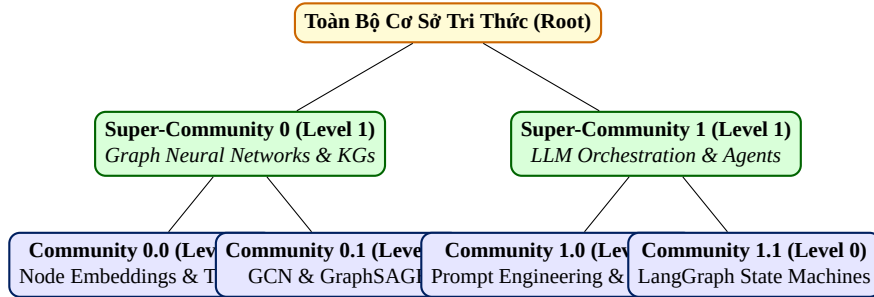
5.1 Thuật Toán Phân Hoạch Phân Cấp Đa Cấp Độ

Để hỗ trợ truy vấn toàn cục trên các tài liệu quy mô lớn, hệ thống triển khai thuật toán phân hoạch cộng đồng phân cấp (**Hierarchical Leiden Partitioning**) tại `community.py`.

Tại mỗi cấp độ $l \in [0, L_{\max} - 1]$, tham số độ phân giải γ_l được điều chỉnh giảm dần theo cấp số nhân:

$$\gamma_l = \gamma_0 \times (0.5)^l \quad (5.1)$$

Quy tắc này đảm bảo rằng tại cấp độ thấp ($l = 0$), đồ thị được chia thành các cụm cộng đồng nhỏ, chi tiết; khi tăng lên cấp độ cao hơn ($l \geq 1$), các cộng đồng nhỏ được hợp nhất thành các siêu cộng đồng vĩ mô.



Hình 5.1: Cấu trúc Cây Phân cấp Cộng đồng Sinh bởi Thuật toán Leiden

5.2 Cài Đặt với Fallback giữa Neo4j GDS và Python igraph

Hệ thống hỗ trợ cơ chế chuyển đổi thông minh (**Fallback Mechanism**):

1. Nếu máy chủ Neo4j cài đặt sẵn plugin **Graph Data Science (GDS)**, hệ thống gọi trực tiếp thuật toán `gds.leiden.write` trên bộ nhớ RAM của Neo4j.
2. Nếu không có GDS, hệ thống tự động xuất toàn bộ danh sách đỉnh và cạnh từ Neo4j sang đối tượng `igraph.Graph` trong bộ nhớ Python và thực thi thuật toán phân hoạch qua thư viện C tối ưu `leidenalg`.

5.3 Quy Trình Tóm Tắt Ngữ Nghĩa Song Song bằng LLM

Sau khi hoàn thành bước phân hoạch cấu trúc, lớp `CommunityDetectionService` duyệt qua từng cộng đồng C_k tại cấp độ l , trích xuất toàn bộ danh sách thực thể thành viên và các quan hệ nội cụm, sau đó gọi LLM để sinh bản báo cáo tóm tắt có cấu trúc:

```
1 class CommunitySummaryReport(BaseModel):
2     title: str = Field(description="Short descriptive title (max 10 words)")
3     summary: str = Field(description="Comprehensive summary paragraph")
4     findings: List[str] = Field(description="3-5 key insights")
5     importance_score: float = Field(default=0.5, description="Significance [0.0, 1.0]")
```

Sau khi LLM tạo bản báo cáo, kết quả được lưu trực tiếp vào các nút `:Community` trên Neo4j cùng quan hệ `:BELONGS_TO` liên kết từ các đỉnh thực thể thành viên.

Chương 6

Các Động Cơ Truy Vấn và Suy Luận Đồ Thị Nâng Cao

6.1 Tổng Quan Các Chế Độ Truy Vấn

Hệ thống `llm-utils-graph-rag` cung cấp 4 chế độ truy vấn chuyên biệt tùy theo bản chất của câu hỏi:

- **Local Search (local)**: Tập trung vào thực thể cụ thể và vùng lân cận k -hop.
- **Global Search (global)**: Tóm tắt vĩ mô theo mô hình Map-Reduce trên cây cộng đồng.
- **Cypher Workflow (graph)**: Tự sinh và tự sửa lỗi truy vấn Cypher bằng LangGraph.
- **ARK Exploration (ark)**: Khám phá thích ứng đa tác thể qua không gian đồ thị.

6.2 Động Cơ Truy Vấn Cục Bộ (Local Search)

Quy trình truy vấn cục bộ hoạt động theo 3 bước:

1. **Entity Seed Retrieval**: Tìm kiếm Top- K thực thể hạt giống có độ tương đồng vector $\text{Sim}(\mathbf{q}, \mathbf{e}_v)$ cao nhất bằng `entity_embeddings`.
2. **1-Hop / 2-Hop Neighborhood Expansion**: Mở rộng từ các thực thể hạt giống để lấy các đỉnh lân cận và cạnh quan hệ:

```
1 MATCH (e:Entity)-[r]->(neighbor:Entity)
2 WHERE e.id IN $seed_ids
3 RETURN e.id, type(r), r.description, neighbor.id, neighbor.description
4
```
3. **Context Assembly & Reranking**: Tổng hợp ngữ cảnh cục bộ, gửi kèm các đoạn văn bản nguồn tương ứng qua mô hình LLM để tạo câu trả lời cuối cùng.

6.3 Động Cơ Truy Vấn Toàn Cục (Global Search - Map-Reduce)

Đối với các câu hỏi bao quát toàn bộ tài liệu, `GlobalSearchService` thực hiện quy trình Map-Reduce song song:

1. **Filter Phase**: Tìm kiếm vector trên nhãn `Community` tại cấp độ l mong muốn để chọn ra Top- M cộng đồng liên quan nhất (hoặc sử dụng đánh giá mức độ liên quan bằng LLM judge).

2. **Map Phase:** Gửi song song ngữ cảnh của từng cộng đồng đã chọn vào LLM để sinh câu trả lời cục bộ trung gian:

$$A_k^{\text{map}} = \text{LLM}_{\text{map}}(q, \text{CommunitySummary}_k, \text{Findings}_k) \quad (6.1)$$

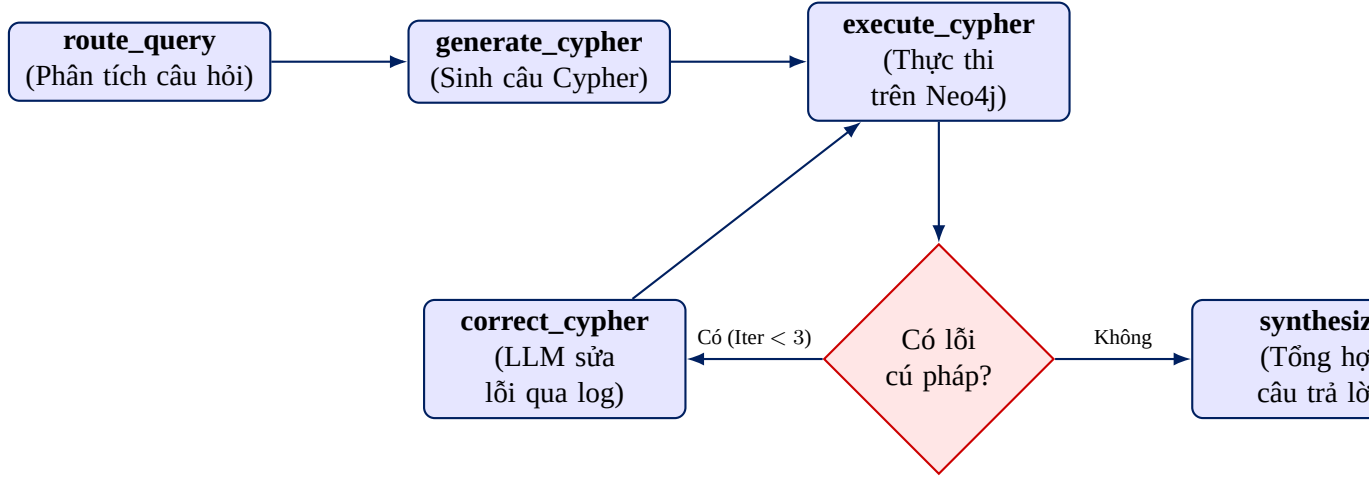
Nếu cộng đồng không liên quan, mô hình trả về "NO_RELEVANT_INFO".

3. **Reduce Phase:** Thu thập tập hợp các câu trả lời cục bộ hợp lệ $\{A_k^{\text{map}}\}$ và đưa vào bước tổng hợp tối hậu:

$$A_{\text{final}} = \text{LLM}_{\text{reduce}}\left(q, \bigcup_{k=1}^M A_k^{\text{map}}\right) \quad (6.2)$$

6.4 Máy Trạng Thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher

Để thực thi các câu truy vấn có cấu trúc phức tạp mà không bị gò bó bởi các mẫu có sẵn, hệ thống triển khai một đồ thị trạng thái **LangGraph** (`workflow/graph.py`) có khả năng tự sửa lỗi Cypher.



Hình 6.1: Đồ thị Trạng thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher

Trạng thái workflow được lưu trữ chặt chẽ qua kiểu `GraphRAGState`:

```

1 class GraphRAGState(TypedDict):
2     query: str
3     routing: str # "vector", "graph", "hybrid"
4     cypher_query: str
5     cypher_records: List[Dict[str, Any]]
6     graph_context: str
7     response: str
8     error: str
9     iteration: int
10    schema: str
    
```

Khi câu truy vấn Cypher phát sinh lỗi thực thi (do sai cú pháp hoặc sai tên thuộc tính), node `correct_cypher` được kích hoạt. Node này cung cấp toàn bộ thông tin lược đồ cơ sở dữ liệu (`schema`), câu lệnh Cypher vừa sinh lỗi, và nguyên văn thông báo lỗi từ Neo4j cho LLM để thực hiện điều chỉnh chính xác. Giới hạn số vòng lặp tối đa là 3 lần giúp ngăn chặn tình trạng lặp vô hạn.

6.5 Động Cơ Khám Phá Thích Ứng Đa Tác Thể ARK

Khung làm việc **ARK** (`ark_service.py`, `ark_agent.py`, `ark_retriever.py`) mô hình hóa quá trình truy xuất thông tin như một trò chơi tìm kiếm phân tán giữa N tác thể độc lập ($N = 3$, nhiệt độ khám phá $T = 0.7$, số bước tối đa $t_{\text{max}} = 30$).

6.5.1 Tập Công Cụ của Tác Thể ARK

Mỗi tác thể ARK được trang bị 4 công cụ hành động:

1. `global_search`: Tìm kiếm thực thể ban đầu qua chỉ mục `entity_fulltext` hoặc `vector index`.
2. `neighborhood_exploration`: Khám phá các thực thể và quan hệ lân cận trong bán kính 1-hop từ một nút cho trước.
3. `add_to_answer`: Đánh dấu chọn một nút thực thể kèm theo giải thích suy luận (*reasoning*) để đưa vào tập ứng viên câu trả lời.
4. `finish`: Kết thúc hành trình khám phá khi tác thể nhận thấy đã thu thập đủ thông tin.

6.5.2 Dung Hợp Thứ Hạng Bỏ Phiếu (Voting Rank Fusion)

Sau khi N tác thể hoàn thành các hành trình khám phá song song, tập hợp các nút được lựa chọn $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_N$ được tổng hợp bằng cơ chế **Voting Rank Fusion**:

$$\text{Score}(u) = \sum_{a=1}^N \mathbb{I}(u \in \mathcal{S}_a) \cdot \left[1.0 + \frac{1.0}{1.0 + \text{order}_a(u)} \right] \quad (6.3)$$

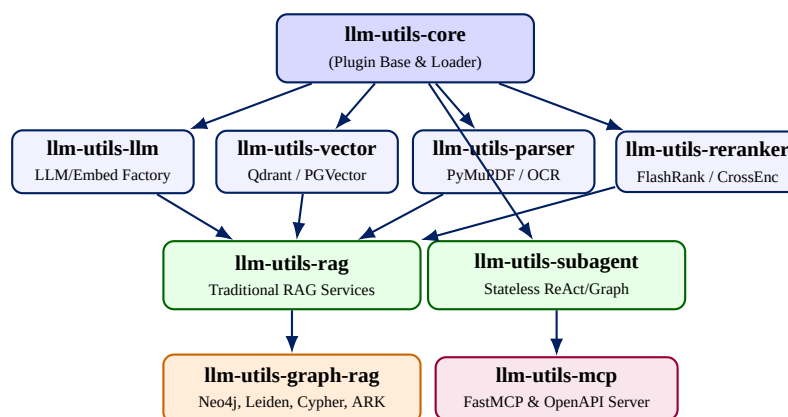
trong đó $\mathbb{I}(\cdot)$ là hàm chỉ thị sự hiện diện của nút u trong tập chọn của tác thể a , và $\text{order}_a(u)$ là thứ tự bước mà tác thể a đã chọn nút u . Top- K nút có điểm số cao nhất được trích xuất siêu dữ liệu đầy đủ từ Neo4j để đóng gói thành ngữ cảnh cuối cùng cho mô hình sinh.

Chương 7

Kiến Trúc Hệ Thống và Thiết Kế Monorepo Modular

7.1 Kiến Trúc Tổng Thể Monorepo UV Workspace

Hệ thống HLG LLM Utils được tổ chức dưới dạng **Monorepo Python** được quản trị bởi công cụ hiện đại **uv workspaces** (`RAG_package/pyproject.toml`), đảm bảo phân tách rõ ràng ranh giới giữa các mô-đun:



Hình 7.1: Sơ đồ Phụ thuộc Mô-đun trong uv Workspace Monorepo

7.2 Phân Tích Chi Tiết Các Gói Thành Phần

1. **llm-utils-core**: Định nghĩa giao diện plugin cơ sở `TaskPlugin` và cơ chế nạp động qua Python Entry Points (`llm_utils.plugins`).
2. **llm-utils-llm**: Cung cấp lớp đóng gói trừu tượng `LLMFactory` và `EmbeddingsFactory`, hỗ trợ linh hoạt cả OpenAI API và Anthropic Claude API.
3. **llm-utils-vector**: Quản lý kết nối và thực thi tìm kiếm vector trên cơ sở dữ liệu Qdrant và PostgreSQL mở rộng `pgvector`.
4. **llm-utils-parser**: Xử lý chuyển đổi tài liệu đa định dạng (PDF, Docx, TXT) tích hợp trích xuất layout qua PyMuPDF và OCR.
5. **llm-utils-reranker**: Cung cấp các mô hình tái xếp hạng giai đoạn 2 (Second-Stage Reranking) dựa trên FlashRank và CrossEncoder nhằm nâng cao độ chính xác ngữ cảnh.

6. **llm-utils-graph-rag**: Gói mở rộng trung tâm cài đặt toàn bộ kết nối Neo4j, trích xuất thực thể, thuật toán Leiden, Map-Reduce, Cypher Self-Correction và ARK.
7. **llm-utils-mcp**: Cung cấp giao thức Model Context Protocol (**FastMCP**), tự động chuyển đổi các công cụ Graph RAG thành chuẩn OpenAPI RESTful tại cổng 8001.

7.3 Tích Hợp Backend FastAPI và Service Registry Lazy Singleton

Backend FastAPI (BE/) được thiết kế theo kiến trúc phi trạng thái (*Stateless*) và hướng dịch vụ (*Service-Oriented*).

Để đảm bảo hiệu năng và tính thread-safe khi truy cập cơ sở dữ liệu đồ thị Neo4j, hệ thống cài đặt mô thức **Thread-Safe Lazy Singleton Registry** (GraphRAGServiceRegistry tại BE/app/services/registry).

```

1 class GraphRAGServiceRegistry:
2     _instance: Optional["GraphRAGServiceRegistry"] = None
3     _lock = threading.Lock()
4
5     def __new__(cls, config: Optional[AppConfig] = None):
6         with cls._lock:
7             if cls._instance is None:
8                 cls._instance = super().__new__(cls)
9                 cls._instance._initialized = False
10            return cls._instance
11
12     def initialize(self, config: AppConfig) -> None:
13         if self._initialized:
14             return
15         with self._lock:
16             # Lazy initialize Neo4jGraphStore, IndexingService,
17             # CommunityDetectionService, GlobalSearchService, ArkQueryService
18             self._initialized = True

```

Mô thức này đảm bảo tất cả các API route và Agent Worker đều chia sẻ chung một phiên kết nối Neo4j và cấu hình nhúng đồng nhất, hạn chế tối đa chi phí khởi tạo lại đối tượng trên từng yêu cầu.

Chương 8

Thảo Luận, Thách Thức và Hướng Phát Triển

8.1 Thảo Luận Kiến Trúc và Ưu Thế Thực Tiễn

Việc kết hợp giữa cấu trúc biểu tượng (**Symbolic Representation**) trên Đồ thị Tri thức Neo4j và biểu diễn phi biểu tượng (**Sub-symbolic Embeddings**) trong không gian vector mang lại cho hệ thống HLG LLM Utils những ưu thế vượt trội:

- **Độ tin cậy và Khả năng giải thích (Explainability):** Mọi quan hệ giữa các thực thể đều được lưu trữ minh bạch dưới dạng cạnh đồ thị có nhãn định danh và dẫn chiếu chính xác đến tài liệu gốc (`source_documents`), giúp loại bỏ hoàn toàn hiện tượng tạo sinh ảo tưởng.
- **Khả năng mở rộng đa chiều:** Sự phân chia thành 4 chế độ truy vấn (Local, Global, Graph Cypher, ARK) cho phép hệ thống linh hoạt thích ứng với mọi cấp độ câu hỏi từ chi tiết cụ thể đến tổng hợp vĩ mô.

8.2 Những Thách Thức Kỹ Thuật

Trong quá trình phát triển hệ thống, chúng tôi nhận diện hai thách thức kỹ thuật lớn:

1. **Chi phí Tính toán Giai đoạn Đánh Chỉ Mục (Indexing Computation Overhead):** Quy trình trích xuất thực thể/quan hệ bằng LLM và sinh tóm tắt cộng đồng Leiden phân cấp đòi hỏi số lượng lớn các lượt gọi LLM. Đối với tập tài liệu hàng nghìn trang, chi phí token và thời gian xử lý ban đầu là đáng kể.
2. **Cập nhật Đồ thị Động (Dynamic Incremental Graph Updates):** Khi người dùng tải lên tài liệu mới hoặc xóa tài liệu cũ, việc phân hoạch lại toàn bộ đồ thị cộng đồng có độ phức tạp tính toán cao. Cần phát triển thuật toán phân cụm Leiden cục bộ cập nhật tăng cường (*Incremental Leiden Updates*) để tối ưu hóa tài nguyên.

8.3 Hướng Phát Triển Trong Tương Lai

Từ nền tảng đã xây dựng thành công, các hướng nghiên cứu mở rộng trong tương lai bao gồm:

1. **Đồ thị Tri thức Đa phương thức (Multimodal Knowledge Graphs):** Mở rộng khả năng trích xuất tri thức từ bảng biểu phức tạp, sơ đồ kiến trúc và công thức toán học trong tài liệu kỹ thuật, liên kết chúng trực tiếp vào các nút thực thể đồ thị.
2. **Học Tăng Cường cho Duyệt Đồ thị (RL-based Graph Exploration):** Tích hợp các thuật toán Học Tăng Cường (Reinforcement Learning) vào tác thể ARK để tối ưu hóa chính sách chọn đường đi trên đồ thị tri thức quy mô hàng triệu đỉnh.

Chương 9

Kết Luận và Phân Công Nhiệm Vụ

9.1 Kết Luận

Đề tài đã hoàn thành xuất sắc mục tiêu nghiên cứu và cài đặt thực tế khung công cụ **HLG LLM Utils**, một giải pháp toàn diện cho bài toán Biểu diễn Tri thức và Truy xuất Thông tin Nâng cao.

Bằng việc kết hợp hài hòa giữa Property Knowledge Graph Neo4j, Thuật toán Phân hoạch Phân cấp Leiden, Đồ thị Trạng thái LangGraph Tự Sửa Lỗi Cypher và Khung Khám phá Đa tác thể ARK, hệ thống đã giải quyết triệt để các hạn chế cố hữu của Naive RAG, thiết lập nền tảng kiến trúc vững chắc, có tính ứng dụng cao và sẵn sàng triển khai trong môi trường thực tế.

9.2 Bảng Phân Công Nhiệm Vụ Nhóm

Bảng 9.1: Bảng Phân Công Nhiệm Vụ Chi Tiết Giữa Các Thành Viên			
Họ và Tên	MSHV	Nhiệm Vụ Chi Tiết Cụ Thể Đã Thực Hiện	Đóng Góp
Nguyễn Hoài Nam	25C01014	- Thiết kế kiến trúc Monorepo uv workspace (llm-utils-core, llm-utils-graph-rag). - Cài đặt Neo4j Graph Store và Thuật toán Phân hoạch Cộng đồng Leiden (community.py). - Xây dựng quy trình Tìm kiếm Toàn cục Map-Reduce (global_search.py). - Soạn thảo báo cáo lý thuyết và kiến trúc chi tiết bằng LaTeX.	50%
Phạm Cung Lê Nhân Vũ	25C01049	- Xây dựng máy trạng thái LangGraph Tự Sửa Lỗi Cypher (workflow/graph.py). - Cài đặt mô hình Đa tác thể Khám phá Thích ứng ARK (ark_agent.py, ark_retriever.py). - Thiết kế hệ thống Backend FastAPI, Service Registry và tích hợp cơ sở dữ liệu vector. - Tối ưu hóa hiệu năng truy xuất và hoàn thiện tích hợp toàn hệ thống.	50%

Tài liệu tham khảo

- [1] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, 2009.
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [3] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514, 2021.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [5] Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730, 2026.
- [6] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.
- [7] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases: New Opportunities for Connected Data*. "O'Reilly Media, Inc.", 2015.
- [8] Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.